Machine Translated by Google

PISCINE OBJECT

Module 00 - Encapsulation

Résumé : Ce document vous présentera un aspect important de la programmation orientée objet : l'encapsulation.

Version : 1.00

Machine Translated by Google

Contenu

je Préambule 2

II Règles générales 3

III Exercice 00 : Diviser pour mieux régner 4

IV Exercice 01 : Je ne sais pas ce que je fais ! 6

Bonus V 8

V.1 Exercice 00 : Diviser pour mieux gouverner 8

V.2 Exercice 01 : Qu'est-ce que je regarde ?! 8

NOUS Soumission et évaluation par les pairs 9

Machine Translated by Google

Chapitre I

Préambule

Dans ce module, vous découvrirez un concept fondamental de la programmation orientée objet.

Arrêtons-nous 5 secondes pour définir un certain nombre de noms qui vous seront utiles dans ce groupe.

- « Objet » : un concept incarnant une « chose ».
- « Instance d'un objet » : Une entité concrète correspondant à un objet donné.
- « Attributs » : Variable interne de l'instance d'un objet
- « Méthode » : une manière de communiquer avec une instance d'un objet, pour lui faire exécuter une action. actes.

Maintenant que nous sommes d'accord sur ces 4 mots...

Voici… Vous l'avez deviné !… L'« Encapsulation » !

Derrière ce nom barbare se cache en réalité un concept assez simple : il existe des choses qui « appartiennent » à un objet, et ces éléments peuvent être accessibles ou non à d'autres personnes.

En programmation orientée objet (POO), nous pouvons définir les attributs et les méthodes de deux manières :

- Privé : ces attributs/méthodes ne seront utilisables que par et strictement par l'objet lui-même
- Public : ces attributs/méthodes peuvent être utilisés/modifiés par l'extérieur de la classe

L'idée est de faire de l'objet une sorte de boîte noire : l'extérieur ne peut accéder qu'à ce qui est public, et rien à ce qui se passe à l'intérieur de la boîte noire, qui serait privé.

Oui, le type Protected existe, mais nous en reparlerons plus tard.
sur

Machine Translated by Google

Chapitre II

Règles générales

- Votre programme ne doit en aucun cas planter (même lorsqu'il manque de ressources). mémoire), et ne devrait pas s'arrêter inopinément.
  Si cela se produit, votre projet sera considéré comme non fonctionnel et votre note sera de 0.

- Vous devez soumettre un Makefile qui compilera vos fichiers sources. Il ne doit pas reconnecter.

- Votre Makefile doit au moins contenir les règles suivantes :
  $(NOM), tout, nettoyer, fclean et re.

- Compilez votre code avec C++ et les options -Wall -Wextra -Werror

Votre code doit être conforme à la norme C++98. Il devrait ensuite compiler. si vous ajoutez l'indicateur -std=c++98.

- Essayez de toujours développer en utilisant le maximum de fonctionnalités C++ possibles (par exemple, choisissez `<cstring>` plutôt que `<string.h>`). Vous pouvez utiliser des fonctions C, mais privilégiez toujours leurs versions C++ lorsque cela est possible.

- Toute bibliothèque externe et les bibliothèques Boost sont interdites.

Machine Translated by Google

Chapitre III

Exercice 00 : Diviser pour mieux régner

	Exercice
Exercice 00 : Diviser pour mieux régner	
Répertoire de dépôt : ex/	
Fichiers à soumettre : ex00/*.cpp, ex00/*.hpp, ex00/main.cpp, Makefile Fonctions autorisées :	
conteneurs et structure STD standard	

Dans cet exercice, vous devez modifier le code contenu dans le fichier DivideAndRule.cpp.
Il contient :

- Un système bancaire
- Un système de comptes clients

Il s'agit d'une banque, dotée d'un système de comptes clients.
Mais ce code a été créé par un développeur qui n'a pas suivi les principes de conception avancée d'objets (Advanced Objects) de la page 42 et qui, par conséquent, ne savait pas comment créer des classes correctement encapsulées ! Tout est accessible à tous !

N'importe qui peut donc modifier le solde de son compte bancaire ! Pratique, non ? Mais pas très logique… Idem pour les prêts : tout le monde peut en obtenir sans rien faire, ni même les rembourser !

Tout cela n'est pas très propre... Alors...

Votre tâche consistera à prendre ce code et à encapsuler correctement les deux classes Account.
et la Banque, afin qu'aucune action ne puisse être entreprise de manière illogique !

Machine Translated by Google

Vous devez satisfaire au moins aux exigences suivantes :

- La banque doit percevoir 5 % de chaque flux de trésorerie entrant sur ces comptes clients.
- Les comptes ne doivent jamais avoir deux identifiants identiques.
- Les attributs des structures ne doivent pas être modifiables de l'extérieur.
- La banque doit pouvoir créer, supprimer et modifier les comptes de ces clients
- La banque doit pouvoir accorder un prêt à un client, dans la limite de ses fonds.
- Il doit être impossible d'ajouter de l'argent au compte d'un client sans passer par le banque
- Si cela est pertinent, la création d'un getter et d'un setter est obligatoire. Les getters copiés ne seront pas acceptés.
- Si cela est pertinent, la création d'un getter constant est obligatoire. Les getters constants copiés ne seront pas acceptés.

Lors de l'évaluation, chaque choix effectué au cours de cet exercice doit être justifié.

Machine Translated by Google

Chapitre IV

Exercice 01 : Je ne sais pas ce que je fais !

	Exercice 01
Exercice 01 : Je ne sais pas ce que je fais !	
Répertoire de dépôt : ex01/	
Fichiers à soumettre : ex01/*.cpp, ex01/*.hpp, ex01/main.cpp, Makefile Fonctions autorisées :	
conteneurs et structure STD standard	

Dans cet exercice, vous devrez appliquer ce que vous avez appris lors de l'exercice précédent. Vous devez créer un conteneur Vector2, qui doit contenir deux composantes X et Y, de type float. Vous devez créer une classe Graph et y placer un point (représenté par la nouvelle structure Vector2).

Une fois tous les points ajoutés, vous devez les afficher à l'écran, via un dessin ASCII.

Réfléchissez à la manière dont ces deux attributs sont encapsulés : souhaitez-vous qu'ils soient privés ? publics ?

Créez une classe Graph, qui doit contenir une taille, dans Vector2, et une liste de Vector2 représentant les différents points du graphique.

Vous devez réfléchir à l'encapsulation de cette classe : - Comment gérerez-vous l'ajout d'un nouveau point ?
- Comment allez-vous afficher le graphique sur la console ?

Machine Translated by Google

Voici un exemple de résultat attendu pour un graphique composé des éléments suivants points :

- 0/0
- 2/2
- 4/2
- 2/4

```
>&5.   . . . . .
>&4.   .X.   . .
>&3.   . . . . .
>&2.   .X.X.
>&1.   . . . . .
>&0X.   . . . .
>&012345
```

Vous devez satisfaire au moins aux exigences suivantes :

- Vous devez être en mesure d'expliquer la décision que vous avez prise concernant l'encapsulation.
- Vector2 doit contenir un attribut représentant x et y.

L'utilisateur doit être autorisé à ajouter un nouveau Vector2 au graphe.

- L'utilisateur doit pouvoir afficher le contenu du graphique sur la console sous une forme similaire à celle d'un graphique.

formulaire

Machine Translated by Google

Chapitre V

Bonus

En guise de bonus, nous proposons des exigences supplémentaires pour les deux exercices

V.1 Exercice 00 : Diviser pour mieux gouverner

- Vous ne pouvez créer aucune autre méthode que des accesseurs constants (getters) dans la structure Account.
- La structure du compte doit être interne à la structure de la banque.

La structure de la banque doit contenir un tableau d'opérateurs permettant d'obtenir un compte par son identifiant, mais il est interdit d'utiliser une boucle while ou for pour trouver le compte correspondant à cet identifiant.

La gestion des erreurs doit se faire via une instruction `throw`, et la fonction principale doit les gérer. erreurs

V.2 Exercice 01 : Qu'est-ce que je regarde ?!

- Création d'une représentation graphique au format PNG de la classe Graph, où tous les Les points seront visibles.
- Ajouter une fonctionnalité de ligne à la classe de graphique
- Lire un fichier d'entrée contenant une liste de points

La partie bonus ne sera évaluée que si la partie obligatoire est PARFAITE. Par « parfaite », on entend que la partie obligatoire a été entièrement réalisée et fonctionne sans aucun dysfonctionnement. Si vous n'avez pas satisfait à TOUTES les exigences obligatoires, votre partie bonus ne sera pas évaluée.

Machine Translated by Google

Chapitre VI

Soumission et évaluation par les pairs

Pour cette évaluation, il sera vérifié que vous avez correctement encapsulé les classes, mais il y aura également des questions sur les raisons de ces modifications et sur les personnes logiques pour lesquelles ces modifications ont été apportées.

L'idée est de vous obliger à RÉFLÉCHIR à ce que vous faites, et non pas simplement à suivre des règles sans bien comprendre pourquoi elles existent.

Veuillez déposer votre travail dans votre dépôt Git comme d'habitude. Seul le contenu de votre dépôt sera évalué lors de la soutenance. N'hésitez pas à revérifier les noms de vos dossiers et fichiers pour vous assurer de leur exactitude.